

Alpha Burritos

Alpha Burritos is a modern, interactive React application developed for CERN's ALPHA experiment. It provides researchers and analysts with a user-friendly interface to browse, visualize, and analyze signal data from the experiment's "burrito detectors"—specialized detection systems that capture antimatter annihilation events.

The application communicates with a custom RESTful API to fetch and display critical data, including JSON parameter files and detector-specific PNG images captured at various timestamps. By consolidating multiple data streams into a single, responsive interface, Alpha Burritos simplifies the complex task of antimatter detection analysis and monitoring.

System Status Expert Mode: ON

acq: running
analysis: running
temperature: running

acquisition START STOP LOGS
analysis START STOP LOGS
temperature START STOP LOGS

red_pitaya_1: ON
red_pitaya_2: ON
red_pitaya_3: ON
red_pitaya_4: ON
red_pitaya_5: ON

[LOG OUT](#)

Offline Analysis Configuration

[POSITRONS](#)
[ANTIPROTONS](#)
[FIT ENABLED](#)
[RE-ANALYSE](#)
[HIDE DETAILS](#)
[GO TO THE GUIDE](#)

Parameter	Value	Parameter	Value
AcqTime	31536000	config	antiprotons
cutoff	5	fft	true
fit	true	hostnames	[object Object]
nPoint	10000	nPoint_pmt11	10000
red_pitaya_1	true	red_pitaya_2	true
red_pitaya_3	true	red_pitaya_4	true
red_pitaya_5	true	run_avg	1
TrigDelay	65.536	TrigVal	4

Mapping			
Red Pitaya	Channel 1	Channel 2	Hostname
1	PDS	LDS	rp-k0a21.local
2	USAT	DSAT	rp-k073b1.local
3	BDS	PMT11	rp-k0b54.local
4	USCT	DSCT	rp-k0b22.local
5	Scope	UDS	rp-k0e4b.local

[Latest acquisition: 2026-03-18 07:26:59 - From the positrons trigger](#)

Data Selection

☒ Auto-refresh
 Year:
 Month:
 Day:
 Timestamp:
 Detector:

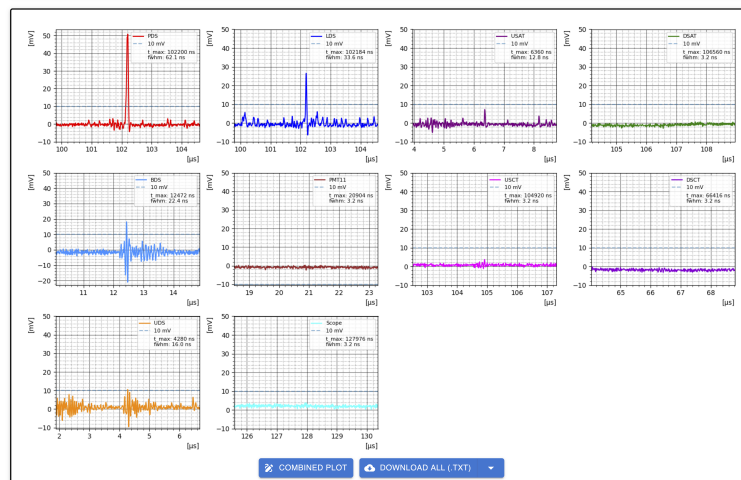
Positrons - 2026-03-17 11:12:19

Location	Area [V·ns]	FWHM [ns]	Peak [mV]	Rise [ns]	Time Peak [ns]	Interval [ns]	Arrival [ns]
PDS	3.37	62.1	53.2	49.5	1.02e+5	8.00	1.02e+5
LDS	1.15	33.6	26.6	25.6	1.02e+5	8.00	1.02e+5
USAT	0.102	12.8	6.96	11.2	6.36e+3	8.00	6.35e+3
DSAT	0.00	3.20	-2.80	0.00	1.07e+5	8.00	1.07e+5
BDS	0.980	22.4	-21.0	14.4	1.25e+4	8.00	1.25e+4
PMT11	0.00	3.20	-2.39	0.00	2.09e+4	8.00	2.09e+4
USCT	0.00	3.20	3.58	3.20	1.05e+5	8.00	1.05e+5
DSCT	0.00	3.20	-3.18	0.00	6.64e+4	8.00	6.64e+4
UDS	0.262	16.0	10.5	12.8	4.28e+3	8.00	4.27e+3
Scope	0.00	3.20	3.78	4.79	1.28e+5	8.00	1.28e+5

[No comment](#)

[SHARE LINK](#)
[REVERSE TABLE](#)
[UPDATE COMMENTS](#)
[DOWNLOAD SIGNAL \(.TXT\)](#)

Detector: PDS



Skimmer

Particle:
 Detector:
 Acquisition timestamp:
 Acquisition timestamp:

☒ Last N acquisitions
 N:

Timestamps	Area [V·ns]	FWHM [ns]	Peak [mV]	Rise [ns]	Time Peak [ns]	Interval [ns]	Arrival [ns]
2026-03-17 16:00:40	0.0962	9.59	6.34	17.6	6.09e+4	8.00	6.09e+4
2026-03-17 16:00:46	0.102	9.17	7.37	9.17	2.19e+3	8.00	2.18e+3
2026-03-17 16:00:50	0.162	16.0	7.58	14.4	1.28e+5	8.00	1.28e+5
Mean	0.120	11.6	7.10	13.7	6.38e+4	8.00	6.38e+4

[DOWNLOAD THE SUMMARY \(.CSV\)](#)

Features

- Temperature Monitoring:** View real-time temperature readings from multiple detector hosts (pitaya boards) in a compact, color-coded table. The temperature data is fetched from the

backend and auto-refreshes every 1 second. The table uses a green background to match the application's button style.

- **Year/Month/Day Selection:** Choose the year, month, and day to filter available data files.
- **Timestamp Selection:** Select a specific acquisition timestamp from available JSON files.
- **Parameter Display:** View key parameters (area, FWHM, peak, rise, time peak, dt, arrival) for each detector.
- **Combined Image View:** See the combined detector image for the selected acquisition.
- **Detector Selection:** Choose a specific detector (PDS, BDS, DSAT, USAT, PMT11, etc.) to view its processed image.
- **Signal Download:** Download the raw signal as a text file for the selected detector and timestamp.
- **Download All Signals:** Download signal files for all available detectors at once.
- **Comments:** Add and edit comments for specific acquisition files.
- **Auto-Refresh:** Automatically refresh the list of available files.
- **Skimmer:** Browse and export a range of acquisitions and parameters as CSV.
- **Responsive UI:** Built with React and styled for clarity and usability.
- **Authentication:** Secure login for authorized users.
- **Choose Configuration:** Select the analysis mode ("positrons" or "antiprotons") and toggle the fit option for the current session.
- **Re-analyse:** Trigger a new analysis of the currently selected file with the current configuration.
- **Script Logs:** View stdout/stderr logs for each Python worker script (acquisition, analysis, temperature) directly in the UI via a dialog, with a refresh button to fetch the latest output.
- **Red Pitaya Status:** In expert mode, displays the connection status of all Red Pitaya devices as color-coded chips (green = ON, red = OFF, grey = DISABLED). The backend pings all enabled hosts in parallel with a 2-second cache to avoid flooding.

Project Structure

```
src/
  main.jsx           # React entry point
  main.css           # Global styles
  app/
    App.jsx          # Main React component
    LoginForm.jsx     # User authentication form
    MainTitle.jsx     # App title and logo
    MessageAlert.jsx  # Temporary top-left Snackbar for
status and error messages
  assets/
    ALPHA_Logo_png.png # ALPHA experiment logo
  configuration/
    ChooseConfiguration.jsx # Select analysis mode
(positrons/antiprotons) and fit option
    ConfigTable.jsx     # Displays configuration details in a
table
  dataDisplay/
    DataDisplay.jsx     # Container for detector data
visualization
    DetectorImage.jsx    # Shows image for a selected detector
```

DetectorImageAll.jsx detectors)	# Shows combined detector image (all
DownloadButton.jsx text file	# Button to download the signal as a
DownloadAllButton.jsx all detectors	# Button to download all signals for
Parameters.jsx JSON	# Displays detector parameters from
Skimmer.jsx acquisitions as CSV	# Browse and export a range of
SkimmerPlotButton.jsx generation for selected data	# Button to trigger skimmer plot
dataSelection/ AutoRefresh.jsx	# Toggle auto-refresh of file list
DataSelection.jsx controls	# Container for data selection
DateSelector.jsx selection	# Dropdowns for year/month/day
DetectorSelector.jsx	# Dropdown for detector selection
TimeStampSelector.jsx selection	# Dropdown for timestamp/file
systemStatus/ SystemStatus.jsx	# System status panel with expert-
mode controls TemperatureDisplay.jsx	# Real-time temperature readings for
detector hosts WorkerMonitor.jsx	# Monitors Python worker process
status PitayaStatus.jsx	# Displays Red Pitaya connection
status	

How It Works

1. Select Year/Month/Day:

Use the dropdowns to pick a year, month, and day. The app fetches available JSON files for that period.

2. Select Timestamp:

Choose a file (timestamp) from the dropdown. The app loads its parameters and images.

3. View Parameters:

The **Parameters** panel displays key values for each detector.

4. View Images:

- An image containing subplots (one signal per subplot) is shown by default.
- Use the "Combined plot" button to switch to a single plot with all signals overlaid.
- Use the detector dropdown to view images for individual detectors.

5. Download Signals:

- Use the "Download the signal" button to download the raw signal for the selected detector.
 - Use the "Download the signals" button to download signals for all available detectors at once.
6. **Add Comments:**
Add notes or observations about specific acquisition files for collaborative research.
 7. **Auto-Refresh:**
Enable auto-refresh to automatically update the list of available files every 2 seconds.
 8. **Skimmer:**
Use the Skimmer tool to select a range of acquisitions, view their parameters in tabular form, and export as CSV.
 9. **Authentication:**
Login is required to access the main features. Credentials are checked via the backend API.
 10. **Choose Configuration:**
Use the "Choose configuration" panel to select the analysis mode ("positrons" or "antiprotons") and toggle the fit option for the current session.
 11. **Re-analyse:**
Use the "Re-analyse" button to trigger a new analysis of the currently selected file with the current configuration.
-

API Endpoints

The React app expects the following backend API (see `server.js`):

- **Test if the API is working:**

`GET /api/test`

- Returns a JSON file to confirm that the API is working.
- Example: `/api/test`
- Response:

```
{ "message": "API is working" }
```

- **List JSON Files:**

`GET /api/:year/:month/:day/json`

- Returns a list of JSON filenames for the specified year, month, and day.
- Example: `/api/2025/06/02/json`
- Response:

```
["data-2025-06-02_08-25-03.json", "file-2025-06-02.json"]
```

- **Get JSON File Content:**

GET /api/json/:jsonFilename

- Returns the parsed JSON content of the specified file.
- Example: /api/json/data-2025-06-02_08-25-03.json
- Response:

```
{ "PDS": { "area": ..., ... }, ... }
```

- **Get Combined Image (single plot, all signals):**

GET /api/Together/:imageName

- Returns the PNG image with all signals overlaid in a single plot (from the **Together** subdirectory).
- Example: /api/Together/data-2025-06-02_08-25-03.png
- Response:
Binary PNG image.

- **Get Subplots Image (default):**

GET /api/Same/:imageName

- Returns the PNG image containing subplots (one signal per subplot) for all signals (from the **Same** subdirectory).
- Example: /api/Same/data-2025-06-02_08-25-03.png
- Response:
Binary PNG image.

- **Get Detector-Specific Image:**

GET /api/img/:detector/:imageName

- Returns the PNG image for a specific detector.
- Example: /api/img/PDS/data-2025-06-02_08-25-03.png
- Response:
Binary PNG image.

- **Get Detector-Specific Signal:**

GET /api/signal/:detector/:jsonFilename

- Returns the signal as a text file for a specific detector and JSON file.
- Example: /api/signal/PDS/data-2025-06-02_08-25-03.json
- Response:
Text file

- **Get Detector-Specific Signal (CSV):**

GET /api/signal/csv/:detector/:jsonFilename

- Returns the signal as a CSV file for a specific detector and JSON file.

- Example: `/api/signal/csv/PDS/data-2025-06-02_08-25-03.json`
- Response:
CSV file

- **Get Comments:**

`GET /api/comments/:jsonFilename`

- Retrieves any saved comments for a specific JSON file.
- Example: `/api/comments/data-2025-06-02_08-25-03.json`
- Response:

```
{ "comment": "This detection shows interesting peak values." }
```

- Returns `{ "comment": null }` if no comment exists.

- **Post Comments:**

`POST /api/comments/:jsonFilename`

- Saves a comment for a specific JSON file.
- Example: `/api/comments/data-2025-06-02_08-25-03.json`
- Request Body:

```
{ "comment": "This detection shows interesting peak values." }
```

- Response:

```
{ "success": true, "message": "Comment updated successfully" }
```

- **Get/Update Configuration:**

- `GET /api/configuration` — Get the current configuration (from `ANALYSIS_DIR/configurations/configuration.json`).
- `POST /api/configuration` — Update the configuration (expects JSON body).

- **Get the Latest Dump Timestamp:**

`GET /api/latest`

- Returns the latest dump timestamp.
- Example: `/api/latest`
- Response:

```
{ "latest": "2025-08-25_11-04-04", "particle": "antiprotons" }
```

- **Re-analyse a JSON File:**

`POST /api/reanalyse/:filename`

- Triggers a re-analysis for the specified JSON file. Requires authentication.
- Example: `/api/reanalyse/data-2025-06-02_08-25-03.json`
- Response:

```
{ "success": true }
```

- **Generate a Skimmer Plot:**

`POST /api/skimmer/:detector`

- Triggers skimmer plot generation for the selected detector. Requires authentication.
- Example: `/api/skimmer/PDS`
- Response:

```
{ "success": true }
```

- **Check Python Worker Status:**

`GET /api/status/:pidfile`

- Checks if a Python worker process is running, based on the given pidfile name.
- Example: `/api/status/acquisition`
- Response:

```
{ "worker": "running" }
```

or

```
{ "worker": "stopped" }
```

- **Authentication:**

- `POST /api/login` — Authenticate user and set the HTTP-only token cookie (expects { `login`, `password` } in body).
- `GET /api/profile` — Returns { `"login": "your_login"` } when the user is authenticated.
- `POST /api/logout` — Logout user and clear the authentication cookie.

- **Start a Python Worker Script (requires authentication):**

`POST /api/start/:name`

- Starts a Python worker script in the background. Allowed names: **acquisition**, **analysis**, **temperature**.
- Example: **/api/start/acquisition**
- Response:

```
{ "success": true, "script": "START_ACQUISITION.py", "pid": 12345 }
```

- Returns **409** with **{ "error": "acquisition is already running" }** if already running.

- **Stop a Python Worker Script (requires authentication):**

POST /api/stop/:name

- Stops a running Python worker script. Allowed names: **acquisition**, **analysis**, **temperature**.
- Example: **/api/stop/analysis**
- Response:

```
{ "success": true, "script": "ONLINE_ANALYSIS.py", "pid": 12345 }
```

- Returns **{ "success": true, "message": "analysis is not running" }** if not running.

- **Get Python Worker Script Logs:**

GET /api/logs/:name

- Returns the log output (stdout + stderr) for a Python worker script. Allowed names: **acquisition**, **analysis**, **temperature**.
- Supports an optional **?lines=N** query parameter to return only the last N lines.
- Example: **/api/logs/acquisition**
- Response:

```
{ "name": "acquisition", "log": "...log content..." }
```

- Returns **{ "name": "acquisition", "log": "", "lines": 0 }** if no log file exists yet.

- **Get Red Pitaya Connection Status:**

GET /api/pitaya-status

- Pings all enabled Red Pitaya hosts in parallel and returns their connection status. Results are cached for 2 seconds.

- Example: `/api/pitaya-status`
- Response:

```
{
  "pitayas": [
    { "name": "red_pitaya_1", "host": "rp-f0a821.local",
      "status": "on" },
    { "name": "red_pitaya_2", "host": "rp-f073bf.local",
      "status": "off" },
    { "name": "red_pitaya_3", "host": "rp-f0bd54.local",
      "status": "disabled" }
  ],
  "updatedAt": "2025-10-08T08:42:16.000Z"
}
```

- **Get Temperature Data:**

GET `/api/temperature`

- Returns the latest temperature readings for all detector hosts.
- Example: `/api/temperature`
- Response:

```
{
  "hosts": {
    "rp-f0a821.local": { "temp_c": 41.59, "error": null },
    "rp-f073bf.local": { "temp_c": 42.82, "error": null }
  },
  "timestamp_utc": "2025-10-08 08:42:16"
}
```

Error Handling

- Returns `404` if a file or image is not found.
- Returns `400` for invalid requests.
- Returns `500` for server errors.

Directory Structure

The API expects files to be organized as:

```
MAIN_DIR/
  YYYY/
    MM/
      DD/
        JSON/
```

```
data-YYYY-MM-DD.json
comments.json
Together/
  data-YYYY-MM-DD.png
Same/
  data-YYYY-MM-DD.png
PDS/
  data-YYYY-MM-DD.png
  data-YYYY-MM-DD.txt
BDS/
  data-YYYY-MM-DD.png
  data-YYYY-MM-DD.txt
...
```

Development

Prerequisites

- Node.js (v18+ recommended)
- npm

Install dependencies

```
npm install
```

Start the React App

Development mode:

```
npm run dev
```

The app will be available at <http://localhost:5173> by default.

For production deployments, it is recommended to use a web server such as **NGINX** to serve the built frontend.

See the [NGINX documentation](#) for setup instructions and configuration examples.

Start the Backend API

You have two options:

Option 1: Run directly with Node.js (recommended for development):

```
node server.js
```

Option 2: Run with PM2 (recommended for production):

```
# Start the server
pm2 start server.js --name burritos

# Save the process list
pm2 save

# Set up PM2 to start on system boot
pm2 startup

# Follow the instructions and run the command output by the previous
step
```

- View logs: `pm2 logs burritos`
- Stop the server: `pm2 stop burritos`
- Restart the server: `pm2 restart burritos`

The API will be available at <http://localhost:3001> by default.

Customization

- **API URL:**

If your backend runs on a different host or port, update the API URLs in the React components (e.g., `Parameters.jsx`, `DetectorImage.jsx`, etc.).

- **Data Directory and Environment Variables:**

The backend expects data in a specific directory structure (see API documentation above).

The main directory of the data and other sensitive configuration must be declared in a `.env` file at the project root.

Example `.env` file:

```
MAIN_DIR=/absolute/path/to/your/data
ANALYSIS_DIR=/absolute/path/to/your/analysis
PORT=3001
USER_LOGIN=your_login
USER_PASSWORD_HASH=your_bcrypt_hash
JWT_SECRET=your_jwt_secret
PYTHON_PATH=/usr/bin/python3
```

- **MAIN_DIR**: Absolute path to the main data directory (required).

- `ANALYSIS_DIR`: Absolute path to the analysis directory (required).
- `PORT`: Port for the backend server (default: 3001).
- `USER_LOGIN`: Username for authentication (required).
- `USER_PASSWORD_HASH`: Bcrypt hash of the password for authentication (required).
- `JWT_SECRET`: Secret key for JWT authentication (required).
- `PYTHON_PATH`: Path to the Python executable used for analysis scripts. If not specified, defaults to `python3`.

Note: Never commit your `.env` file to version control as it may contain sensitive information.

Useful Resources

- [React Documentation](#)
 - [Vite Documentation](#)
 - [Express Documentation](#)
 - [Node.js Documentation](#)
 - [PM2 documentation](#)
 - [NGINX Documentation](#)
 - [JavaScript Reference \(MDN\)](#)
 - [JSON Format](#)
 - [RESTful API Design](#)
 - [GitHub Guides](#)
 - [ALPHA Experiment \(CERN\)](#)
-

This project uses React, Vite, and Express. For backend API details, see `server.js`.

Developed by Samuel Niang (Swansea University) and Adriano Del Vincio (Brescia University)